

Constrained Self-Service Group Formation on Serverless NoSQL Infrastructure: A Deployed Reference Architecture and Experience Report

Mithun Gowda B^a, Harsha N^a, Naren V^a, Nevil Anson Dsouza^a
NV Sushma^b, A S Manjunatha^{b*}

^aDepartment of Computer Science and Engineering,

Don Bosco Institute of Technology (Affiliated to VTU), Kumbalagodu, Bangalore 560074, India

^bDepartment of Chemistry,

Don Bosco Institute of Technology (Affiliated to VTU), Kumbalagodu, Bangalore 560074, India

*Guide and corresponding author email: madhumanjuas@dbit.co.in

Abstract

Self-service web platforms that let end users form their own groups under composition rules must enforce those rules at the moment of user action, maintain cross-document invariants on document-store infrastructure, and operate inside the quotas of third-party services. This paper is an experience report on a reference architecture for that problem, deployed in production for an engineering cohort with an enrolment roster of 823 students and released as open source.

We contribute three things. First, a parameterised composition model whose mandatory-representation constraint is *disjunctive*, matching the rule the deployment actually enforced, together with a feasibility theorem reduced to three independent conditions. In establishing it we correct an error in our earlier formulation, which asserted a condition that is not in fact necessary, and we note that the general case with overlapping requirement groups is a minimum hitting set problem and therefore not checkable in polynomial time. Second, a two-phase validation design that separates recoverable warnings from irrecoverable blocks during iterative group building, with its complexity and, more importantly, the precise scope of its correctness guarantee stated explicitly. Third, a candid account of what this architecture got wrong. We avoided multi-document transactions although the underlying database provides them, and we placed rate-limiting and quota-spreading state in per-instance memory on a serverless platform where such state does not survive instance recycling. A successor deployment by the same authors reverted the first of these decisions.

We also withdraw a technique reported in an earlier version of this work, which spread outbound email across several free-tier accounts to multiply an effective daily quota. Multiplying a per-account quota in that manner conflicts with the intent of the provider’s terms, and we set out the sanctioned alternatives instead. Finally, we retract the previously reported figure of 100% constraint satisfaction for completed groups: because group completion is gated by the very predicate being measured, that figure is true by construction and carries no information. We state which quantities the deployment genuinely instrumented and which it did not, so that the gap is visible rather than concealed. The implementation comprises 10,937 lines of TypeScript across 58 files and is archived at a fixed commit.

Keywords: Self-service web platforms; Group composition constraints; NoSQL data integrity; Serverless architecture; Experience report; Negative results

1 Introduction

1.1 Problem Statement

Modern web platforms increasingly adopt self-service models in which end users, rather than administrators, perform complex, multi-step workflows such as group formation, resource allocation, and collaborative project setup. These workflows impose *configurable constraints*: rules that govern the composition of user-formed groups, the distribution of attributes across members, and the conditions under which a group reaches a valid terminal state. Examples include hackathon team composition rules (“at least one designer and one backend developer”), organizational unit staffing policies (“no more than four members from any single department”), and marketplace listing requirements (“minimum two product categories represented”).

Enforcing such constraints in real time, providing immediate feedback as each member is added to an evolving group, is substantially more challenging than batch validation after group formation is complete. The system must evaluate constraint feasibility at every intermediate state, distinguish between violations that can self-correct through subsequent additions (soft warnings) and violations that are irrecoverable (hard blocks), and handle concurrent modifications from multiple users operating on the same group simultaneously.

These challenges are compounded on **NoSQL document-store infrastructure**, which trades the ACID transaction guarantees of relational databases for horizontal scalability and real-time subscription capabilities [1–3]. Without multi-document transactions, enforcing cross-document invariants, such as “each user belongs to at most one group”, requires architectural patterns that are absent from conventional web application frameworks.

A third, often-overlooked challenge arises from **third-party API rate limits**. Self-service platforms that integrate external services (email delivery, SMS verification, payment processing) must operate within per-key quotas imposed by service providers. During peak usage, such as a registration burst where hundreds of users authenticate within hours, a single API key’s daily quota can be exhausted rapidly, causing service degradation precisely when demand is highest.

1.2 Research Gap

Existing web systems address these challenges in isolation but not in combination:

- **Real-time web architectures** [4, 5] enable push-based data delivery via WebSocket [6] and Server-Sent Events (SSE) but do not address constraint enforcement or data integrity.
- **Constraint satisfaction systems** [7, 8] provide formal frameworks for constraint modeling but operate in batch mode, not as real-time web services with concurrent user interactions.
- **NoSQL consistency models** [9–11] characterize trade-offs between consistency, availability, and partition tolerance but do not prescribe application-level patterns for enforcing multi-document invariants.
- **Serverless computing platforms** [12–14] provide auto-scaling infrastructure but introduce challenges for stateful operations (rate limiting, session management) due to ephemeral function instances.
- **Self-service web platforms** [15, 16] enable user-driven collaboration but typically lack formal constraint models and comprehensive data integrity guarantees.

Each of these strands is mature in isolation. What we did not find, when building the system described here, was a published account of how they compose in a single deployed platform that must answer to real users under a real deadline. That is the gap this paper addresses, and we are careful about its nature: it is a gap in reported engineering experience, not a gap in available techniques. Every mechanism we use is individually known. Fig. 1 sketches the region of the design space our deployment occupies; we stress that the axes are qualitative and that

the placement of other approaches reflects their design intent as documented by their authors, not measurements we performed.

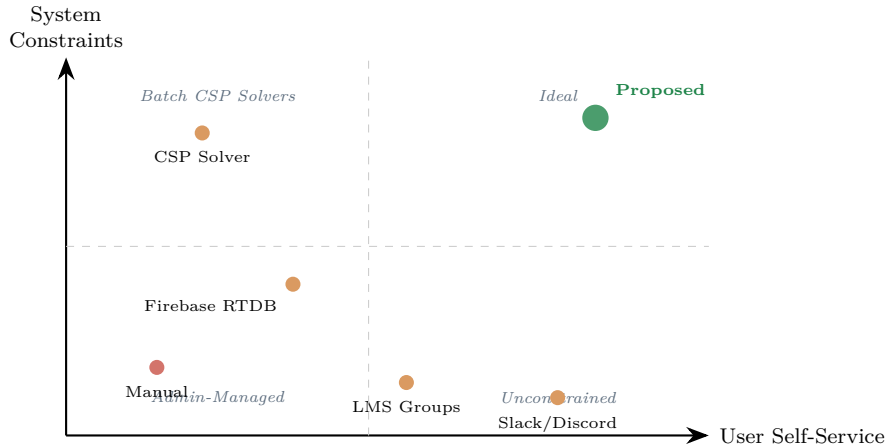


Figure 1: Positioning of web group-formation approaches along user self-service and system constraint enforcement dimensions. Existing solutions cluster in three quadrants; the upper-right quadrant (high self-service + high constraint enforcement) remains unoccupied.

1.3 Scope and Contributions

This is an experience report. We make no claim that the individual mechanisms described here are new, and we have removed such claims from an earlier version of this work. Its value, if it has any, lies in the fact that the system was built, deployed, used, and is available for inspection, and that we are willing to describe its defects as precisely as its merits.

1. A **composition model and feasibility result** (Section 4). The model admits disjunctive mandatory-representation requirements, which is what the deployment actually enforced and what our earlier conjunctive formulation failed to capture. We reduce feasibility to three independent conditions, correct a non-necessary condition asserted in our earlier formulation, and identify the boundary beyond which feasibility checking becomes NP-hard.
2. A **two-phase validation design and its honest correctness scope** (Section 4). We state what the two phases do and do not guarantee. In particular we make explicit that the guarantee is conditional on serialised application of the validator, which the original deployment did not ensure.
3. An **account of three design decisions we now consider mistaken** (Sections 6, 7, and 9): avoiding multi-document transactions that the database provides, holding rate-limiter state in per-instance memory on a serverless runtime, and spreading email across multiple free-tier accounts. For each we give the reasoning at the time, the consequence, and what we would do instead.

What this paper does not claim. It does not claim a novel constraint-satisfaction algorithm; the validator is a linear scan over a small member list. It does not claim generality beyond its single deployment. It does not claim measured superiority over the alternative approaches discussed in Section 8, and we have removed the scored comparison that appeared in an earlier version because we had no methodology capable of supporting it.

The remainder of this paper is organized as follows. Section 2 surveys related work across seven research areas. Section 3 presents the system model and design principles. Section 4 formalizes the constraint model with feasibility and correctness proofs. Section 5 describes the system architecture, including the round-robin API key rotation. Section 6 details the six-layer

data integrity framework. Section 7 presents the security architecture. Section 8 reports the production evaluation results. Section 9 discusses findings and limitations. Section 10 outlines future work. Section 11 concludes.

2 Related Work

2.1 Real-Time Web Architectures

The evolution from polling-based to push-based web architectures has enabled a new class of interactive applications. The WebSocket protocol [4] provides full-duplex communication over a single TCP connection, enabling server-initiated data delivery without the overhead of HTTP long-polling. Lubbers and Greco [5] demonstrate that WebSocket reduces latency by 3:1 and bandwidth by 500:1 compared to HTTP polling for high-frequency data streams. Server-Sent Events (SSE) [17] provide a simpler, unidirectional alternative suitable for scenarios where the server pushes updates to clients without requiring bidirectional communication.

Firebase Firestore [18] implements a document-level subscription model through its `onSnapshot` API, which pushes data changes to connected clients whenever the subscribed document is modified. Unlike WebSocket-based architectures that require explicit server-side event emission, Firestore’s subscription model is declarative: clients subscribe to document paths, and the infrastructure handles change detection, conflict resolution, and delivery. This model is particularly well-suited for applications where multiple users observe shared state (e.g., group composition) and require immediate visual feedback when that state changes.

However, real-time delivery infrastructure alone does not address the *semantic* challenge of constraint enforcement. Existing real-time web frameworks provide the communication substrate but leave application-level invariants, such as group composition rules, membership uniqueness, and state-machine transition validity, entirely to the application developer. Our work builds on Firestore’s real-time capabilities while adding a formal constraint layer and multi-level integrity guarantees.

2.2 Automated and Self-Service Group Formation

Group formation has an established literature, and it is important to be clear about where our model sits within it. Lappas et al. [19] formalise team formation over a social network and establish its NP-hardness; Anagnostopoulos et al. [20] extend this to an online setting in which tasks arrive over time. Both treat formation as an *optimisation* problem solved *on behalf of* the participants. Sun et al. [21] report a deployed two-phase procedure for an engineering cohort of 238 students, combining graph clustering with simulated annealing under balance and isolation penalties, and release verification artefacts.

Our problem is deliberately weaker, and this is the honest answer to the question of what differentiates it. We do not optimise an objective and we do not assign anyone to a group. Users choose, and the system’s only job is to decide, in constant time and at the instant of a click, whether a proposed addition can still lead to a valid group. Consequently our feasibility result is not a competitor to the optimisation results above: it is a much simpler admissibility question, and we claim only that stating it precisely is useful for implementers, not that it is deep. Where prior work asks which partition is best, we ask which single addition is permissible. We are not aware of a prior statement of necessary and sufficient parameter conditions for this admissibility question in the disjunctive-requirement form of Section 4, but we regard the result as elementary and would not be surprised to find it as folklore.

2.3 Constraint Satisfaction in Web Systems

Lappas et al. [19] formalize team formation as a constrained optimization problem on social networks and establish its NP-hardness for the general case where both team quality and inter-member communication cost must be optimized simultaneously. Their approximation algorithms provide provable guarantees for offline settings where all candidates are known in advance.

Anagnostopoulos et al. [20] extend this to *online* settings where members arrive sequentially and assignments are irrevocable, a model closer to self-service web platforms where users register and join groups over extended periods. Their competitive analysis shows that online algorithms achieve ratios within constant factors of offline optima for practical constraint structures.

Wi et al. [22] apply genetic algorithms to optimize team competency coverage, demonstrating that evolutionary approaches handle complex multi-objective constraints in R&D settings.

The constraint satisfaction programming (CSP) literature [7, 8, 23] provides the formal foundations for constraint modeling, including arc consistency, backtracking search, and constraint propagation. However, CSP solvers operate in batch mode, they compute complete solutions after all variables and constraints are specified, and are not designed for the interactive, incremental constraint evaluation required by real-time web platforms. Our constraint model draws on CSP formalism (parameterized rules, feasibility analysis) while adapting the evaluation strategy for real-time, per-operation validation.

2.4 NoSQL Data Integrity

Brewer’s CAP theorem [9], formalized by Gilbert and Lynch [10], establishes that distributed systems cannot simultaneously guarantee consistency, availability, and partition tolerance. NoSQL databases such as Amazon DynamoDB [11] and Google Cloud Firestore [18] prioritize availability and partition tolerance, offering eventual consistency [2] or strong consistency within bounded scopes (e.g., single-document reads in Firestore).

The absence of multi-document ACID transactions in many NoSQL systems creates a fundamental challenge for web applications that must enforce cross-document invariants. Cattell [1] surveys NoSQL data stores and identifies the lack of referential integrity enforcement as a key trade-off. Stonebraker [24] argues that this trade-off is acceptable for workloads that are partitionable by entity, but problematic for workloads requiring cross-entity consistency.

Chang et al. [25] describe Bigtable’s single-row transaction model, which provides atomicity for operations within a single entity but not across entities. Firestore extends this with single-document atomicity and limited multi-document transactions (up to 500 documents per batch), but cross-collection invariants still require application-level enforcement.

Our six-layer data integrity framework (Section 6) addresses this gap by implementing invariant enforcement at multiple application levels, UI guards, API-layer validation, and database security rules, providing defense-in-depth integrity without relying on database-level transactions.

2.5 Serverless Architecture Patterns

Jonas et al. [12] introduce the serverless computing paradigm, arguing that auto-scaling, pay-per-use cloud functions democratize access to scalable infrastructure. The Berkeley view on serverless computing [26] identifies key challenges: cold-start latency, limited execution duration, and the absence of persistent local state.

Baldini et al. [13] survey serverless computing patterns and identify stateful serverless applications as an open problem. McGrath and Brenner [27] benchmark serverless function performance across providers, finding that cold-start latencies range from 100ms to several seconds depending on runtime and provider.

Schleier-Smith et al. [14] argue that the next phase of serverless computing must address state management, proposing programming models that integrate state into the serverless paradigm.

Our architecture operates within the constraints of current serverless platforms: in-memory state (rate-limit counters) is tolerated as best-effort and resets on cold starts; persistent state resides entirely in the NoSQL database; and API key rotation is stateless modulo a single in-memory counter. This design achieves zero-cost deployment on free tiers while accepting the trade-offs inherent in serverless infrastructure.

2.6 Rate Limiting and API Key Management

Rate limiting is a foundational mechanism for protecting web services from abuse and ensuring fair resource allocation. The token bucket and leaky bucket algorithms [28] are widely deployed in API gateways and reverse proxies. The IETF has proposed standardized rate-limit response headers [29] to communicate quota status to clients.

Raghavan et al. [30] address distributed rate limiting in cloud environments, proposing algorithms that maintain global rate limits across geographically distributed enforcement points. Their work focuses on *incoming* request rate limiting (protecting the server), whereas our round-robin algorithm addresses *outgoing* request rate limiting (operating within external API quotas), a complementary but distinct problem.

The specific challenge of multiplying effective API capacity through multi-key rotation has received little formal treatment in the literature. Commercial platforms (SendGrid, Brevo, Mailgun) document per-key quotas but do not provide client-side rotation algorithms. Our round-robin approach (Section 5.5) formalizes this pattern with complexity analysis and fault-tolerance properties.

2.7 Self-Service Platforms and Workflow Management

Self-service web platforms enable end users to perform workflows previously requiring administrator intervention. Van der Aalst et al. [31] survey business process management systems, identifying workflow patterns that recur across domains. Harel’s statecharts [32] provide a visual formalism for modeling complex system behavior, including concurrent states and hierarchical decomposition.

Finite state machines (FSMs) are widely used to model entity lifecycle in web applications [33]. Our system employs FSMs for two critical state transitions: group status (forming $\rightarrow$ full $\rightarrow$ locked) and invitation status (pending $\rightarrow$ approved/rejected/expired). The FSM model ensures that terminal states are immutable and that concurrent transition attempts are handled idempotently.

Bafoutsou and Mentzas [15] classify collaborative systems by functionality, identifying group formation, awareness, and communication as core capabilities. Olson and Olson [16] study distance collaboration, emphasizing the importance of shared awareness of group state, a requirement our real-time subscription architecture directly addresses.

2.8 Web Application Security

The OWASP Top Ten [34] identifies injection, broken authentication, and cross-site scripting (XSS) as the most critical web application security risks. Barth et al. [35] provide robust defenses against cross-site request forgery (CSRF), including the `SameSite` cookie attribute that our implementation employs.

JWT [36] provides a compact, self-contained token format for session management. Hardt [37] defines the OAuth 2.0 framework for delegated authorization. Stuttard and Pinto [38] and Zalewski [39] provide comprehensive treatments of web application attack vectors and of the browser behaviours that defences must account for.

Our five-layer security model (Section 7) implements defense-in-depth [34] with OTP-based authentication [40], JWT sessions, database-level access control, transport security headers,

and IP-based rate limiting. This architecture is designed for environments where institutional single sign-on (SSO) is unavailable, a common constraint in platforms serving users who lack pre-existing organizational credentials.

3 System Model

3.1 Design Principles

The platform is designed around five principles derived from web systems research:

1. **User self-service:** Users initiate group creation, select members, and choose which groups to join, enabling autonomous workflow completion without administrator intervention.
2. **Automated constraint enforcement:** Every member addition is validated against configurable composition rules in real time, preventing invalid group configurations at the point of user action rather than through post-hoc administrative review.
3. **Progressive guidance:** Soft warnings guide users toward constraint satisfaction during intermediate group-building stages, with hard blocks only at the point of violation or group completion. This design respects human decision-making processes by providing information at the point when it becomes actionable.
4. **Data integrity by design:** Duplication prevention, idempotent operations, and cascade cleanup are built into every state transition, not bolted on after the fact. Invariants are enforced at multiple application levels (defense-in-depth).
5. **Platform governance:** Gate controls, bulk data management, real-time statistics, and intervention capabilities provide administrative oversight without micromanagement of individual user workflows.

3.2 Workflow Model

The group formation process follows four phases (Fig. 2):

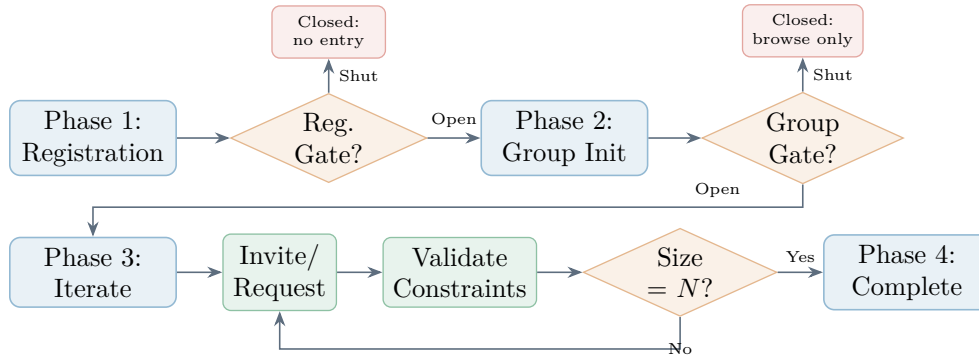


Figure 2: Four-phase group formation workflow. Every decision node carries both outcomes: each administrative gate either admits the flow (Open) or holds it (Shut), and the size test branches on Yes and No. The connector from the group gate to Phase 3 is routed through the gap between the two rows; in an earlier version it was drawn with a right-angle path that crossed over the Phase 3 blocks, which made it appear to terminate at Phase 4.

Phase 1 (Registration): Users authenticate via email OTP and register on the platform. The system validates user identifiers against an enrollment database using configurable patterns. An admin-controlled registration gate enables phased enrollment.

Phase 2 (Group Initiation): Users create new groups (generating a unique human-readable group code) or browse existing open groups. A separate group formation gate controls

when this phase becomes available. The group creator is assigned as the “group lead” with invite and management privileges.

Phase 3 (Iterative Building): This is the core formation phase with two parallel interaction modes:

- **Invite mode:** Group leads invite specific users by identifier. The system validates existence, registration status, group membership status, and pending-invite status before creating the invitation.
- **Request mode:** Users browse open groups, view current composition and constraint status, and submit join requests. The system validates membership and pending-request status before creating the request.

Every member addition, whether via invite acceptance or request approval, passes through the constraint engine (Section 4) and the data integrity framework (Section 6) before being committed to the database.

Phase 4 (Completion): When a group reaches N members with all constraints satisfied, it is marked complete. The full composition validation (Algorithm 2) ensures every constraint is met.

3.3 Dual-Interaction Model

A distinctive feature of the platform is its **dual-interaction model**: group leads actively recruit (invite mode) while unaffiliated users proactively seek groups (browse/request mode). This addresses the “cold-start problem” where early-registering users have few groups to browse. Group leads can seed their groups through invitations while simultaneously receiving join requests from browsers.

The browse interface displays each group’s current composition, attribute distribution, open slots, and constraint status, enabling informed decision-making. Users can see which groups need members with their attribute category (addressing mandatory representation requirements), creating a marketplace-like dynamic where supply (users) and demand (group slots with specific attribute needs) naturally equilibrate.

3.4 Platform Governance Layer

The administrative interface provides three categories of control:

1. **Gate management:** Independent toggles for the registration gate and group formation gate, enabling phased rollouts.
2. **Data management:** CSV bulk upload of user enrollment data, individual user management, and group intervention capabilities (remove members, dissolve groups, transfer group lead).
3. **Real-time monitoring:** Statistics dashboard showing registration counts, group counts by status, attribute distribution across groups, unaffiliated user counts, and constraint violation alerts.

4 Formal Constraint Model

4.1 Definitions and Parameters

Definition 1 (Group and Attribute Model). Let $T = \{m_1, m_2, \dots, m_n\}$ be a group where each member m_i has an attribute category $b(m_i) \in \mathcal{B}$ for a finite set of attribute categories $\mathcal{B}$. The constraint model is parameterized by:

- $N \in \mathbb{Z}^+$: Required group size
- $K \in \mathbb{Z}^+$: Maximum members from any single attribute category, where $K < N$

- $D \in \mathbb{Z}^+$: Minimum number of unique attribute categories represented, where $D \leq |\mathcal{B}|$
- $\mathcal{S} = \{S_1, \dots, S_q\}$: a family of non-empty, pairwise disjoint *requirement groups* with $S_j \subseteq \mathcal{B}$. Each S_j must be represented by at least one member. A singleton $S_j = \{\beta\}$ expresses a mandatory category; a larger S_j expresses a disjunctive requirement such as “at least one member from ECE or EEE”.

We define the following functions over a group T :

Attribute distribution:

$$\delta(T, \beta) = |\{m_i \in T : b(m_i) = \beta\}| \quad (1)$$

Diversity measure:

$$U(T) = |\{\beta \in \mathcal{B} : \delta(T, \beta) > 0\}| \quad (2)$$

A group T is *valid* if and only if all four constraints are satisfied:

$$\mathcal{C}_1(T) : |T| = N \quad (3)$$

$$\mathcal{C}_2(T) : \max_{\beta \in \mathcal{B}} \delta(T, \beta) \leq K \quad (4)$$

$$\mathcal{C}_3(T) : U(T) \geq D \quad (5)$$

$$\mathcal{C}_4(T) : \forall S_j \in \mathcal{S}, \exists m_i \in T : b(m_i) \in S_j \quad (6)$$

$$\text{valid}(T) \iff \mathcal{C}_1 \wedge \mathcal{C}_2 \wedge \mathcal{C}_3 \wedge \mathcal{C}_4 \quad (7)$$

A note on the disjunctive form, and on a divergence between this model and our implementation. An earlier version of this paper defined $\mathcal{C}_4$ conjunctively, over a set $\mathcal{R} \subseteq \mathcal{B}$ of individually mandatory categories. That formulation did not describe the system we deployed. The deployed rule was “at least one member from ECE or EEE”, a single disjunctive requirement, whereas the conjunctive reading of $\mathcal{R} = \{\text{ECE}, \text{EEE}\}$ demands one member of each. The model above is corrected to match the implemented predicate. We record this because it is the kind of drift that formalisation is supposed to prevent and, in our case, did not: the proof in Section 4 was sound with respect to the written model and silently irrelevant to the running code.

We also withdraw the claim, made in an earlier version, that the model is domain-agnostic and reconfigurable “without code changes”. In the released artifact N , K and D are default values of TypeScript function parameters and the requirement group is a pair of hard-coded string literals; there is no administrative interface for any of them. The model is parameterised on paper. The implementation is not. Making the parameters configuration-driven is straightforward and remains undone.

4.2 Constraint Feasibility

Not all parameter combinations produce feasible group configurations. The following theorem establishes necessary and sufficient conditions for the existence of at least one valid group.

Theorem 2 (Feasibility Conditions). *Let $\mathcal{S}$ be non-empty and pairwise disjoint with $q = |\mathcal{S}|$. A group T satisfying $\mathcal{C}_1 \wedge \mathcal{C}_2 \wedge \mathcal{C}_3 \wedge \mathcal{C}_4$ exists if and only if*

$$K \cdot |\mathcal{B}| \geq N \quad (8)$$

$$D \leq \min(N, |\mathcal{B}|) \quad (9)$$

$$q \leq \min(N, |\mathcal{B}|) \quad (10)$$

Proof. Necessity. (8): By $\mathcal{C}_2$ no category contributes more than K members, so $|T| \leq K|\mathcal{B}|$; with $\mathcal{C}_1$ this forces $K|\mathcal{B}| \geq N$. (9): $U(T) \leq \min(|T|, |\mathcal{B}|) = \min(N, |\mathcal{B}|)$, so $D > \min(N, |\mathcal{B}|)$ makes $\mathcal{C}_3$ unsatisfiable. (10): The requirement groups are pairwise disjoint, so the q members witnessing $\mathcal{C}_4$ occupy q distinct categories and are distinct members. Hence $q \leq U(T) \leq \min(N, |\mathcal{B}|)$.

Sufficiency. Let $r = \max(D, q)$; by (9) and (10), $r \leq \min(N, |\mathcal{B}|)$. Choose one category $\beta_j \in S_j$ for each j , which yields q distinct categories by disjointness, and extend this choice with $r - q$ further distinct categories drawn from $\mathcal{B}$, which is possible because $r \leq |\mathcal{B}|$. Place exactly one member in each of these r categories. This uses $r \leq N$ members, satisfies $\mathcal{C}_4$ by construction, and gives $U = r \geq D$, satisfying $\mathcal{C}_3$. Each category now holds one member, and $K \geq 1$, so $\mathcal{C}_2$ holds. Distribute the remaining $N - r$ members greedily into any categories that hold fewer than K members. Such capacity exists throughout: the unused capacity after the seeding step is $K|\mathcal{B}| - r \geq N - r$ by (8). The result satisfies $\mathcal{C}_1$ through $\mathcal{C}_4$. $\square$

Remark 3 (Corrections to our earlier formulation). An earlier version of this paper stated five conditions rather than three, and two of them should not have been there.

1. The condition $K \geq \lceil N/|\mathcal{B}| \rceil$ is not independent: for integral K it is equivalent to (8), since $K \geq \lceil N/|\mathcal{B}| \rceil \iff K \geq N/|\mathcal{B}| \iff K|\mathcal{B}| \geq N$. Listing both overstated the content of the theorem.
2. The condition $|\mathcal{R}| \leq N$ is implied by (10) and needs no separate statement.
3. The condition $|\mathcal{R}| \leq D$ was *not necessary*, and its stated proof was wrong. That proof read $\mathcal{C}_3$ as permitting “only D unique categories”. But $\mathcal{C}_3$ is the lower bound $U(T) \geq D$, not an upper bound, so a group may freely contain more than D distinct categories. A counterexample suffices: $N = 4$, $K = 2$, $D = 1$, $\mathcal{B} = \{a, b, c\}$ and $\mathcal{S} = \{\{a\}, \{b\}, \{c\}\}$ gives $q = 3 > D = 1$, yet T with categories a, b, c, c satisfies all four constraints. We are grateful to the reviewer who noticed the redundancy, which led us to find the error.

Remark 4 (Where this becomes hard). Pairwise disjointness of $\mathcal{S}$ is what makes (10) simple, because it fixes the number of categories needed to witness $\mathcal{C}_4$ at exactly q . If requirement groups may overlap, the smallest number of categories that meets every requirement is the size of a minimum hitting set for $\mathcal{S}$, and deciding feasibility therefore embeds an NP-hard problem [41]. Our deployment used a single requirement group, so this distinction never arose in practice, but a configurable implementation of the general model would need either a restriction to disjoint groups or an exponential check.

The system validates these feasibility conditions at configuration time and alerts administrators to infeasible parameter combinations before any user interaction begins.

4.3 Two-Phase Validation Algorithm

The constraint engine implements a two-phase validation strategy.

Phase A: Pre-Addition Validation. Before every member addition, Algorithm 1 checks whether adding a candidate would violate any constraint. Hard constraints ($\mathcal{C}_1$, $\mathcal{C}_2$) block immediately. Soft constraints ($\mathcal{C}_3$, $\mathcal{C}_4$) produce warnings during intermediate stages but become hard blocks when adding the N -th member.

Phase B: Completion Validation. When the N -th member is added, Algorithm 2 enforces all four constraints simultaneously.

Proposition 5 (Correctness). *If every member addition passes Algorithm 1 and the final group passes Algorithm 2, then the resulting group T satisfies $\text{valid}(T)$.*

Proof. Algorithm 1 enforces $\mathcal{C}_1$ (lines 2 to 4: rejects additions to full groups) and $\mathcal{C}_2$ (lines 5 to 6: rejects additions exceeding the per-category limit K) at every addition. When $|T_a| + 1 = N$, it additionally enforces $\mathcal{C}_3$ and $\mathcal{C}_4$. Algorithm 2 re-checks all four constraints against the final group state. Since both must pass, $\mathcal{C}_1 \wedge \mathcal{C}_2 \wedge \mathcal{C}_3 \wedge \mathcal{C}_4$ holds for the completed group. $\square$

Algorithm 1 Pre-Addition Constraint Validation (`canAddMember`)

Require: Approved members T_a , candidate category β_c , params $N, K, D, \mathcal{S}$ **Ensure:** (valid, errors, warnings)

```
1:  $E \leftarrow \emptyset$ ;  $W \leftarrow \emptyset$ 
2: if  $|T_a| \geq N$  then
3:    $E.add(\text{"Group full"})$ 
4:   return (false,  $E$ ,  $W$ )
5: end if
6: if  $\delta(T_a, \beta_c) + 1 > K$  then
7:    $E.add(\text{"Max } K \text{ from category } \beta_c")$ 
8: end if
9: if  $|T_a| + 1 = N$  then
10:  {Final member: enforce all constraints}
11:   $T' \leftarrow T_a \cup \{\text{candidate}\}$ 
12:  if  $U(T') < D$  then
13:     $E.add(\text{"Need } \geq D \text{ unique categories"})$ 
14:  end if
15:  for each  $S_j \in \mathcal{S}$  do
16:    if  $\sum_{\beta \in S_j} \delta(T', \beta) = 0$  then
17:       $E.add(\text{"Need a member from } S_j")$ 
18:    end if
19:  end for
20: else if  $|T_a| + 1 \geq N - 2$  then
21:   $T' \leftarrow T_a \cup \{\text{candidate}\}$ 
22:  for each  $S_j \in \mathcal{S}$  do
23:    if  $\sum_{\beta \in S_j} \delta(T', \beta) = 0$  then
24:       $s \leftarrow N - |T_a| - 1$ 
25:       $W.add(\text{"No member from } S_j \text{ yet; } s \text{ slots remaining"})$ 
26:    end if
27:  end for
28: end if
29: return ( $|E| = 0$ ,  $E$ ,  $W$ )
```

Algorithm 2 Full Group Composition Validation (`validateComposition`)

Require: Group members T , params $N, K, D, \mathcal{S}$ **Ensure:** (valid, errors)

```
1:  $E \leftarrow \emptyset$ 
2:  $T_a \leftarrow \{m \in T : m.status = \text{"approved"}\}$ 
3: if  $|T_a| \neq N$  then
4:    $E.add(\text{"Group size must be } N")$ 
5: end if
6: Compute  $\delta(T_a, \beta)$  for all  $\beta \in \mathcal{B}$  via hash map
7: if  $\max_{\beta} \delta(T_a, \beta) > K$  then
8:    $E.add(\text{"Max } K \text{ from any category exceeded"})$ 
9: end if
10: if  $U(T_a) < D$  then
11:    $E.add(\text{"Need } \geq D \text{ unique categories"})$ 
12: end if
13: for each  $S_j \in \mathcal{S}$  do
14:   if  $\sum_{\beta \in S_j} \delta(T_a, \beta) = 0$  then
15:      $E.add(\text{"Missing requirement group } S_j")$ 
16:   end if
17: end for
18: return ( $|E| = 0$ ,  $E$ )
```

The scope of this proposition, stated plainly. Proposition 5 is conditional on its hypothesis, and the hypothesis is stronger than it looks. It presumes that additions are applied one at a time, each validated against the state that the previous one produced. A deployment that evaluates two additions concurrently against the same observed state satisfies the letter of each individual check while violating the conclusion, because the validator was never shown the interleaved state. The proposition is therefore a statement about a serialised sequence of additions, not about a concurrent system, and it does not by itself establish anything about the deployed platform. Section 6 describes the mechanism we used instead of serialisation, and Section 9 explains why we now consider that choice a mistake.

4.4 Complexity Analysis

Both algorithms operate in $O(|T| + \sum_j |S_j|)$ time. The attribute distribution computation requires one pass over the member list ($O(|T|)$) using a hash map, and the mandatory representation check inspects each requirement group once ($O(\sum_j |S_j|)$). Since both $|T| = N$ and $\sum_j |S_j| \leq |\mathcal{B}|$ are small constants in practice (typically $N \leq 10$, $|\mathcal{B}| \leq 15$), the validation executes in constant time, enabling sub-50ms client-side execution.

The space complexity is $O(|\mathcal{B}|)$ for the attribute distribution hash map.

4.5 Progressive Warning System

A key design choice is the distinction between **hard errors** and **soft warnings**:

- **Hard errors** (group full, category concentration exceeded) block the addition immediately. These constraints are violated at the point of addition and cannot self-correct through subsequent additions.
- **Soft warnings** (insufficient diversity, missing mandatory category) are surfaced as advisory messages during intermediate stages. They become hard errors only when adding the final member, because the group can still satisfy these constraints through subsequent additions.

The warning threshold is configurable: in our implementation, warnings activate when $|T_a| + 1 \geq N - 2$ (within 2 slots of completion), providing early notice without disrupting early group building.

4.6 Race Condition Protection

In a concurrent multi-user environment, two users may simultaneously attempt to join a nearly-full group. Without protection:

- **Size overflow:** Both additions succeed, producing a group with $N + 1$ members.
- **Constraint violation:** Both additions individually pass validation, but their combined effect violates a constraint.

We employed a **read-before-write** pattern: immediately before writing any member addition, the system re-reads the current group document and re-executes `canAddMember()` against the latest state. This adds one database read per operation.

This does not guarantee correctness, and we should not have claimed that it does. An earlier version of this paper asserted that the pattern “guarantees constraint correctness under concurrent modification”. It does not. Re-reading narrows the window between the check and the write; it does not eliminate it. Two requests may both read a group with $N - 1$ approved members, both evaluate `canAddMember()` favourably, and both write, yielding $N + 1$ members. The probability of this is low because the window is short, which is precisely why the defect is dangerous: it does not appear in testing and it does not appear in a deployment of a few hundred users, so an absence of observed violations is weak evidence. Our claim of zero violations in

Section 8 should be read in that light. It is consistent with the race never being triggered and equally consistent with the race being triggered and never detected, because the deployment had no invariant checker that would have noticed.

The correct mechanism was available the whole time. Firestore provides multi-document transactions with optimistic concurrency control, retrying the transaction body when a document read inside it is modified before commit [42, 43]. Wrapping the read, the check and the write in one transaction makes the check-to-write interval atomic and reduces this entire discussion to a library call. Our reasons for not doing so were, in retrospect, poor ones: an assumption that transactions would be costly on a free tier, and an over-reading of the observation that single-document reads are strongly consistent, which is true and irrelevant, since the hazard lies in the gap between operations rather than within either one. Lamport’s formulation of sequential consistency [44] makes the distinction precise: per-operation consistency constrains what a single read may return, and says nothing about the atomicity of a read-modify-write sequence. Section 9 records what we did about this subsequently.

5 System Architecture

5.1 Three-Tier Serverless Architecture

The platform implements a three-tier serverless architecture (Fig. 3) designed for zero-infrastructure deployment on free-tier cloud services.

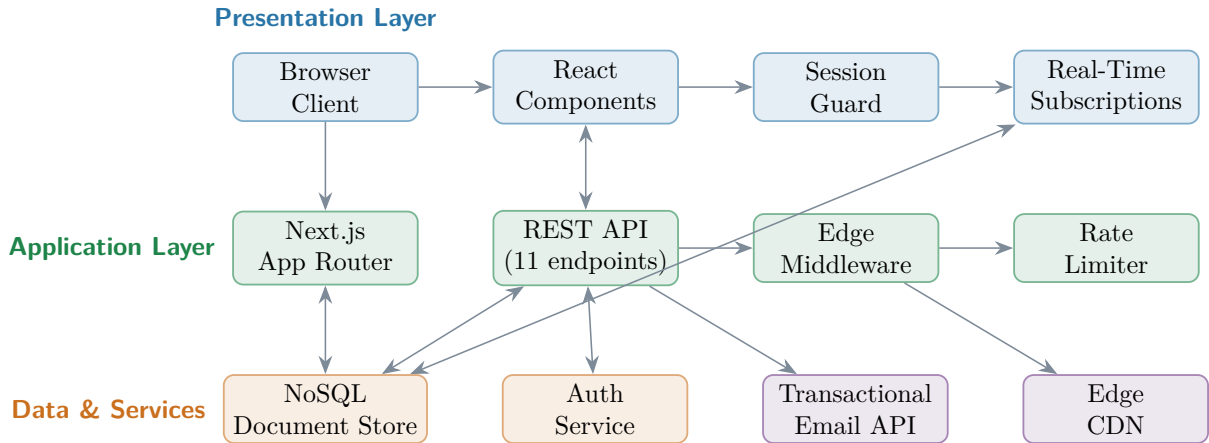


Figure 3: Three-tier serverless architecture with real-time document subscriptions enabling instant constraint feedback and notifications.

The architecture is technology-agnostic in principle; our reference implementation uses Next.js [45], Firebase Firestore [18], and Brevo Email API, but any equivalent stack could implement the same model.

Presentation Layer. The client is a React single-page application rendered via server-side rendering (SSR) for initial page loads and client-side navigation thereafter. The SessionGuard component implements a three-phase authentication check: (1) local storage for cached session state, (2) JWT and database token initialization, and (3) document verification against the registration database. Real-time notifications use the document store’s subscription API with an initialization flag to suppress notification alerts during initial data hydration.

Application Layer. The App Router handles 11 REST API endpoints (Table 1), each implementing input validation, authentication verification, business logic, and response formatting. Edge middleware intercepts all API requests for origin verification, security header injection, and route protection. The rate limiter operates at the IP level with configurable sliding windows.

Data Layer. The NoSQL document store provides real-time subscriptions. The authentication service provides custom token infrastructure for client-side database access. External services include the transactional email API (with multi-key round-robin) and the edge CDN for static asset delivery.

5.2 API Design

The API follows REST conventions in the sense established by Fielding [46] and applied pragmatically in the style described by Richardson and Ruby [47]: resources are addressed by path, state transitions are expressed with HTTP verbs, and no session state is held server-side beyond the signed token.

Table 1 lists all 11 REST endpoints with their methods and purposes.

Table 1: REST API Endpoints

Endpoint	Method	Purpose
/auth/send-otp	POST	Send OTP via email
/auth/verify-otp	POST	Verify OTP, issue JWT
/auth/lookup-usn	POST	Validate user identifier
/auth/logout	POST	Clear session
/auth/firebase-token	GET	Refresh database token
/admin/upload-csv	POST	Bulk data import
/admin/config	GET/POST	System configuration
/admin/students/:id	PUT/DEL	Individual user mgmt.
/admin/reset-database	POST	Database reset (admin)
/notify/email	POST	Email notifications

Each endpoint follows a consistent request/response pattern:

- **Input validation:** Request bodies are validated for required fields and type correctness before any business logic executes.
- **Authentication:** Protected endpoints verify JWT session tokens (user endpoints) or admin ID tokens (admin endpoints).
- **Rate limiting:** Abuse-sensitive endpoints (OTP send/verify) enforce per-IP sliding-window rate limits.
- **Response format:** All responses use JSON with a consistent `{success, error, data}` structure.

5.3 NoSQL Schema Design

The database uses 7 collections (Table 2). A key design decision is **embedding member arrays within group documents** rather than using normalized junction collections. This denormalized design enables:

- **Single-document reads:** Complete group state (all members, attributes, statuses) is retrieved in one database read, critical for real-time constraint validation.
- **Atomic member operations:** Adding or removing a member is an atomic update to a single document, preventing partial-state issues.
- **Efficient subscriptions:** Real-time listeners subscribe to one document per group, reducing subscription overhead.
- **Natural deduplication:** Member arrays are mapped by user identifier, preventing duplicate entries at the data model level.

Table 2: Database Collections and Document Schemas

Collection	Document Key	Schema Fields
users	User ID	name, email, phone, category, section, importedAt
registrations	User ID	name, email, category, section, groupId (FK), groupRole, registeredAt
groups	Group code	name, leadId, members[] (embedded), memberCount, status (FSM), categoryDistribution, isPublic, timestamps
invites	Invite code	type (invite request), groupId, fromId, toId, status (FSM), timestamps
notifications	Auto-generated	userId, type (10 event types), title, message, groupId, fromId, read, createdAt
config	Singleton	registrationsOpen, groupFormationOpen, maxGroupSize, minCategories, maxSameCategory, requiredCategories
otp_codes	Auto-generated	email, otp, expiresAt, used, attempts, createdAt
admins	Email hash	Admin whitelist

The **groups** collection embeds member arrays with the following per-member fields: user identifier, name, attribute category, section, status (**approved** | **pending_invite** | **pending_request**), and join timestamp. The **categoryDistribution** field is a denormalized map (category $\rightarrow$ count) maintained atomically with member operations, enabling $O(1)$ constraint checks without iterating the member array.

5.4 Real-Time Event Architecture

The platform implements real-time data delivery through two complementary mechanisms:

Document subscriptions. Client-side components subscribe to specific documents (the user’s registration document, the user’s group document) and receive push updates whenever these documents change. This enables instant UI updates when a member joins, an invitation is accepted, or the group status transitions.

Query-based polling. For aggregate views (browse page, admin dashboard), the platform uses periodic query-based data fetching rather than document subscriptions, as subscribing to all group documents would create excessive subscription overhead. A **useCallback**-based fetch pattern with session synchronization ensures data consistency:

Listing 1: Session synchronization pattern

```

1 const fetchData = useCallback(async () => {
2   // Re-fetch registration to fix stale session
3   const regDoc = await getDoc(
4     doc(db, "registrations", session.userId)
5   );
6   if (regDoc.exists()) {
7     const data = regDoc.data();

```

```

8 // Sync session if groupId changed
9 if (data.groupId !== session.groupId) {
10     updateSession(data.groupId, data.groupRole);
11 }
12 }
13 // Fetch group and pending invites...
14 }, [session.userId]);

```

The notification system delivers events through two channels simultaneously: (1) in-app real-time delivery via document subscriptions with an `isInitialLoad` flag to distinguish between initial data hydration and genuine new notifications, and (2) email notifications via the transactional email API for offline awareness.

5.5 Outbound Email Under a Provider Quota: A Technique We Withdraw

Free-tier transactional email services limit daily volume, in our provider’s case to 300 messages per day per account [48]. Our registration flow sends one verification code per authentication attempt, so a cohort of several hundred users concentrated into a short window exceeds that allowance.

What we did was to configure the deployment with the credentials of three separate free accounts and rotate between them, so that the aggregate daily allowance became 900 messages. An earlier version of this paper presented that arrangement as a contribution and described it as multiplying effective capacity by a factor of n . We withdraw it, and we think the objection is worth stating in full rather than quietly deleting the section.

Why we withdraw it. A per-account quota is the provider’s mechanism for pricing and for abuse control. Holding several free accounts in order to sum their allowances defeats that mechanism, and the fact that it is technically easy is not an argument that it is legitimate. The provider’s own documented routes to a higher allowance are to upgrade the plan, or, for genuinely separate tenants, to use administrator-managed sub-accounts whose credits are allocated from a single purchased pool rather than added to it [48]. Neither route multiplies a free allowance, which is the point. We also note the practical fragility: nothing in the arrangement is agreed with the provider, so it may be withdrawn without notice, and a platform whose authentication path depends on it has a single point of failure outside its own control and outside its contract.

What should be done instead. For a burst of this shape there are three defensible options. The first is to pay: at the volumes in question the paid tier costs less per month than the engineering time spent building the rotation. The second is to shape the load rather than the quota, by queueing verification messages and draining the queue at the permitted rate, accepting a delay at peak in exchange for staying inside the allowance; for a registration window measured in hours rather than seconds this is usually invisible to users. The third is to remove the dependency, by opening the registration window in scheduled cohorts so that demand never exceeds the allowance in the first place, which costs nothing and is what we would now do.

What we retain from the section. The failover behaviour is worth keeping independently of the quota question, because a transactional email provider can fail for reasons that have nothing to do with quotas. Algorithm 3 is therefore retained below as a description of what the deployment did, and as a reasonable pattern for distributing load across credentials a deployment is legitimately entitled to use, such as the sub-accounts of a paid plan or the endpoints of two independent providers used for redundancy. It should not be read as a recommendation to accumulate free accounts.

The algorithm has several important properties:

Load distribution. The round-robin counter ensures that successive requests use different API keys, distributing load evenly across all n keys. Over m requests, each key receives approximately m/n requests, achieving near-optimal load balancing.

Algorithm 3 Round-Robin API Key Rotation with Failover

Require: Recipient email, subject, body

Ensure: Email sent successfully or all keys exhausted

```
1:  $creds \leftarrow \text{parseCredentials}(\text{env.MULTI\_KEYS})$ 
2: if  $|creds| = 0$  then
3:   throw “No API keys configured”
4: end if
5:  $startIdx \leftarrow rrIndex \bmod |creds|$  {Round-robin selection}
6:  $rrIndex \leftarrow rrIndex + 1$  {Advance for next call}
7: for  $attempt \leftarrow 0$  to  $|creds| - 1$  do
8:    $idx \leftarrow (startIdx + attempt) \bmod |creds|$ 
9:    $result \leftarrow \text{sendEmail}(creds[idx], \text{recipient}, \text{subject}, \text{body})$ 
10:  if  $result.ok$  then
11:    return {Success}
12:  end if
13:  if  $result.status \in \{429, 401, 500\}$  then
14:    continue {Try next key}
15:  end if
16: end for
17: throw “All API keys exhausted”
```

Automatic failover. If a key returns an error (HTTP 429 rate-limited, 401 unauthorized, or 500 server error), the algorithm advances to the next key without manual intervention. In the worst case, all n keys are attempted before failure.

Statelessness, and its cost. The only state is an in-memory counter, which resets whenever the serverless runtime recycles the instance. We described this as acceptable on the grounds that the counter serves load balancing rather than correctness. That is true as far as it goes, but on a platform that creates a fresh instance per cold start the counter is frequently reset to its initial value, so selection is biased towards the first credential rather than being uniform. The distribution we claimed was therefore not the distribution we got, and we did not measure the one we got.

Complexity. The expected-case time complexity is $O(1)$ (the first key succeeds). The worst-case complexity is $O(n)$ (all keys are tried before success or failure). Space complexity is $O(n)$ for the credential array.

Applicability. The failover logic is not specific to email. It applies wherever a client legitimately holds several credentials for an external service, for instance two independent providers configured for redundancy, or the sub-accounts of a paid plan. The qualifier “legitimately” is doing real work in that sentence and is the reason the preceding discussion exists.

Credentials are supplied in one environment variable as a comma-separated list of **key:sender** pairs, parsed at startup into an array, so the set of credentials is a deployment-time choice rather than a compile-time one.

5.6 Unambiguous ID Generation

Group codes and invite codes are generated using a character set designed to prevent visual ambiguity:

ABCDEFGHIJKLMNPQRSTUVWXYZ23456789

This 30-character alphabet excludes five commonly confused characters: I/1/l, O/0. Group codes follow the format **TEAM-XXXX** ($30^4 = 810,000$ combinations) and invite codes follow **INV-XXXXXX** ($30^6 \approx 729$ million combinations), providing sufficient uniqueness for institutional-scale deployments while remaining easy to communicate verbally.

The design is stateless, no sequence counters or coordination is required, and collision probability is negligible for the expected deployment sizes. For a deployment with 200 groups, the birthday-paradox collision probability for 4-character codes is approximately $200^2/(2 \times 810,000) \approx 2.5\%$, mitigated by checking for existing codes before assignment.

6 Data Integrity Framework

A critical challenge in self-service web platforms is **preventing data duplication and enforcing cross-document invariants** without relying on multi-document transactions. When hundreds of users concurrently create groups, send invitations, and process join requests, numerous integrity violations can occur. Our system implements a **six-layer data integrity framework** (Fig. 4).

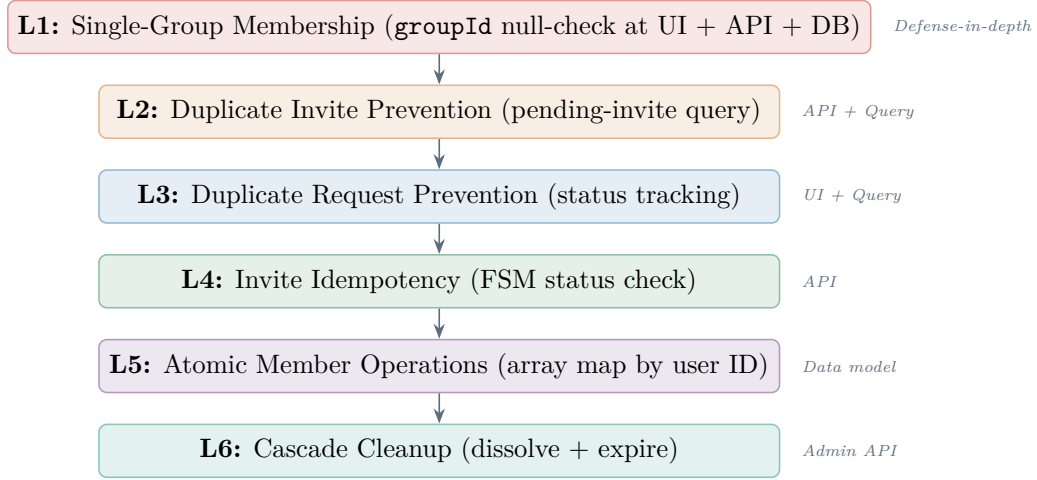


Figure 4: Six-layer data integrity framework. Each layer prevents a specific class of data integrity violation.

We now define the formal invariants and describe the enforcement mechanisms for each layer.

6.1 L1: Single-Group Membership Enforcement

Definition 6 (Single-Membership Invariant). For all users u in the system, u is assigned to at most one group at any time:

$$\forall u \in \mathcal{U}, |\{g \in \mathcal{G} : u \in g.\text{members} \wedge u.\text{status} = \text{approved}\}| \leq 1 \quad (11)$$

This invariant is enforced at **three independent levels**:

1. **UI guards:** Client-side components conditionally render “already assigned” messages and disable join/create buttons when the user’s `groupId` is non-null.
2. **API validation:** The application logic checks the target user’s registration document and rejects operations if `groupId` is set.
3. **Database rules:** Document-level security rules restrict `groupId` updates to the user themselves or a verified group lead of the user’s group.

This defense-in-depth approach ensures that even if client-side validation is bypassed (e.g., via browser developer tools), the server-side and database-level checks reject unauthorized membership changes.

6.2 L2: Duplicate Invite Prevention

Before a group lead issues an invitation, the system executes three sequential checks:

1. **Existing member check:** Verifies the target is not already in the group’s member array.
2. **Pending invite check:** Queries the `invites` collection for any document matching (`toId`, `groupId`, `type="invite"`, `status="pending"`). Rejects if a pending invite exists.
3. **Cross-group check:** Reads the target’s registration document and verifies `groupId` is null.

6.3 L3: Duplicate Join-Request Prevention

Symmetrically, when a user requests to join a group, the system queries for any existing document matching (`fromId`, `groupId`, `type="request"`, `status="pending"`). The browse UI maintains three visual states per group card: default (“Request to Join” enabled), pending (“Request Pending” disabled), and rejected (“Request Declined” with visual indicator).

6.4 L4: Invite Idempotency

Each invite document transitions through a finite state machine (Fig. 5):

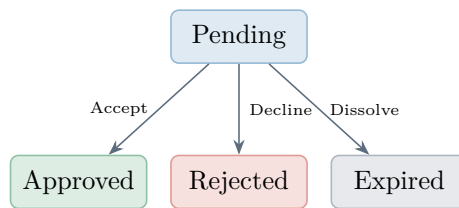


Figure 5: Invite status FSM. Terminal states (approved, rejected, expired) are immutable, no further transitions are permitted.

Once an invite leaves the `pending` state, it cannot be re-processed. The response handler verifies the current status before any action, returning an error for non-pending invites. This guarantees that concurrent acceptance attempts (e.g., clicking “Accept” in two browser tabs) cannot result in a user being added twice.

6.5 L5: Atomic Member-List Operations

Group member arrays use **map operations** (not append) for all modifications:

- **Member approval:** Maps over the array, updating the matching user ID entry’s status. If the ID is not found, the operation fails.
- **Member removal:** Filters the array to exclude the matching user ID.
- **Invariant:** At most one array entry exists per user ID at any time.

6.6 L6: Cascade Cleanup on Group Dissolution

When a group is dissolved, the system performs cascade cleanup:

1. All approved members have their `groupId` and `groupRole` reset to null.
2. All pending invites and requests are marked as `expired`.
3. The group document status is updated to “dissolved.”
4. Notification documents are created for all affected members.

Without cascade cleanup, dissolved groups leave **orphaned state**: users whose registrations still reference a deleted group cannot join new groups because L1 sees a non-null `groupId`.

6.7 Comparison with ACID Properties

Table 3 compares the integrity guarantees of our framework with those provided by traditional ACID transactions in relational databases.

Table 3: Comparison of Integrity Guarantees: ACID vs. Six-Layer Framework

Property	ACID (RDBMS)	Six-Layer (NoSQL)
Atomicity	Multi-row transactions	Single-document atomic updates (L5)
Consistency	Referential integrity constraints	Application-level invariant checks (L1 to L4)
Isolation	Serializable/snapshot isolation	Read-before-write + FSM idempotency (L4)
Durability	Write-ahead log	Firestore persistence guarantees
Cross-entity invariants	Foreign key constraints	Defense-in-depth at UI + API + DB (L1)
Cascade operations	ON DELETE CASCADE	Application-level cascade cleanup (L6)

The six-layer framework achieves comparable integrity guarantees to ACID transactions for the specific invariants required by self-service group formation, without the scalability trade-offs of distributed multi-document transactions.

6.8 Integrity Framework Summary

Table 4 summarizes the complete framework with enforcement mechanisms.

Table 4: Data Integrity Invariants and Enforcement Mechanisms

Invariant	Layers	Enforcement Mechanism
Single-group membership	UI + API + DB	<code>groupId</code> null-check at 3 levels
No duplicate invites	API + Query	Pending-invite query + member check
No duplicate requests	UI + Query	Pending-request query + status tracking
Invite idempotency	API	FSM status check before transition
No duplicate members	Data model	Array map (not append) keyed by ID
Cascade on dissolve	Admin API	Bulk reset + invite expiration

7 Security Architecture

The security architecture implements a five-layer defense-in-depth model (Fig. 6), designed for environments where users lack pre-existing SSO credentials.

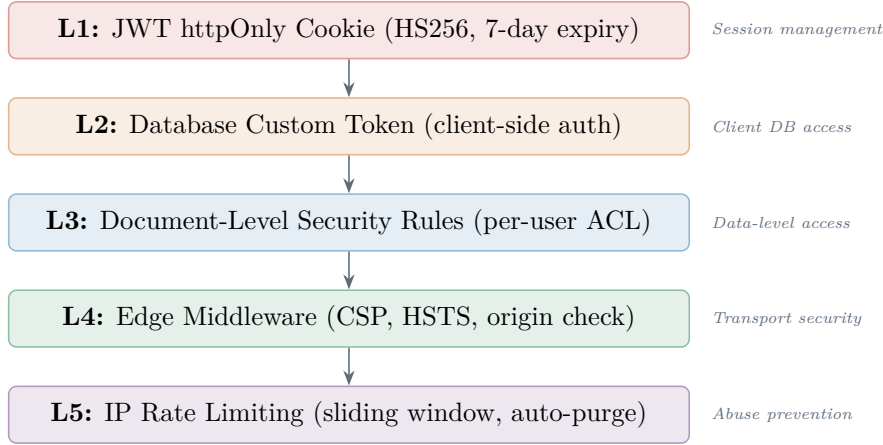


Figure 6: Five-layer defense-in-depth security model. Each layer provides independent protection; compromise of any single layer does not breach the system.

7.1 Layer 1: JWT Session Management

Upon successful OTP verification, the server issues a JWT [36] signed with HS256. The token is stored as an `httpOnly`, `Secure`, `SameSite=Lax` cookie with a 7-day expiry. The `httpOnly` flag prevents JavaScript access, mitigating XSS token theft. The `SameSite` flag prevents CSRF attacks [35].

The JWT payload contains the user’s identifier, name, email, attribute category, and current group assignment, enabling client-side rendering of personalized UI without additional database reads.

7.2 Layer 2: Database Custom Token

The verification endpoint generates a database custom token using the server-side admin SDK. This token enables direct client-to-database communication through the database’s native authentication system, providing real-time subscriptions without routing all reads through the API layer.

7.3 Layer 3: Document-Level Security Rules

Database security rules enforce per-document access control:

Listing 2: Document-level security rules (excerpt)

```

1 match /registrations/{userId} {
2   allow read: if request.auth != null;
3   allow create: if request.auth != null
4     && request.auth.uid == userId;
5   allow update: if request.auth != null
6     && (request.auth.uid == userId
7       || isGroupLead(resource.data.groupId));
8 }
9 match /otp_codes/{doc} {
10  allow read, write: if false; // server-only
11 }
  
```

Key rules: users can only modify their own registration documents (except group leads updating group-related fields for their members); OTP codes are completely inaccessible from clients.

7.4 Layer 4: Edge Middleware

The edge middleware intercepts all requests matching protected route patterns and applies:

- **Route protection:** Dashboard and group creation routes require valid JWT; unauthenticated users are redirected to the registration page.
- **Origin verification:** API routes verify the `Origin` header against an allowlist of permitted origins, preventing unauthorized cross-origin access.
- **Security headers:** Every response includes Content-Security-Policy (restricting script sources and connect destinations), X-Frame-Options (DENY), X-Content-Type-Options (nosniff), Referrer-Policy, Permissions-Policy, and Strict-Transport-Security (HSTS with 1-year max-age) [34].

7.5 Layer 5: IP-Based Sliding-Window Rate Limiting

The rate limiter implements a sliding window algorithm with per-endpoint configuration:

Table 5: Rate Limiting Configuration

Target	Limit	Window
OTP send per IP	5 requests	15 min
OTP verify per IP	10 attempts	15 min
Per-code incorrect attempts	5 attempts	Code lifetime
Per-email OTP cooldown	1 OTP	60 sec

The implementation stores timestamps in an in-memory map with a 5-minute auto-purge cycle. IP extraction follows a priority chain: the platform’s trusted forwarded header, then `x-real-ip`, then the connection’s remote address.

Listing 3: Sliding-window rate limiter

```
1 function rateLimit(ip, action, max, windowMs) {
2   const key = `${ip}:${action}`;
3   const now = Date.now();
4   const entry = store.get(key) || { timestamps: [] };
5   // Filter to timestamps within window
6   entry.timestamps = entry.timestamps
7     .filter(t => now - t < windowMs);
8   if (entry.timestamps.length >= max) {
9     return { allowed: false,
10      retryAfterMs: windowMs - (now - entry.timestamps[0])
11    };
12  }
13  entry.timestamps.push(now);
14  store.set(key, entry);
15  return { allowed: true, retryAfterMs: 0 };
16 }
```

7.6 Threat Model

Table 6 summarizes the threat model and corresponding mitigations.

7.7 OTP Authentication Flow

The email OTP authentication [40] follows a six-step flow:

Table 6: Threat Model and Mitigations

Threat	Layer(s)	Mitigation
Session hijacking	L1	httpOnly + Secure + SameSite cookies
XSS token theft	L1, L4	httpOnly prevents JS access; CSP restricts script sources
CSRF	L1, L4	SameSite cookie; origin verification
Brute-force OTP	L5	5 attempts/code; 5 sends/IP/15min
Direct DB manipulation	L2, L3	Custom token auth; per-document rules
Clickjacking	L4	X-Frame-Options: DENY
MIME sniffing	L4	X-Content-Type-Options: nosniff
Replay attack	L1	JWT expiration (7 days)
Unauthorized API access	L4	Origin allowlist verification

1. User enters their email address.
2. System validates the email exists in the enrollment database.
3. System generates a 6-digit OTP, stores it with a 10-minute TTL and 5-attempt limit.
4. System sends the OTP via the transactional email API with round-robin key rotation (Algorithm 3).
5. User enters the OTP within the validity window.
6. System verifies the OTP, issues a JWT cookie and database custom token, and creates/updates the registration document.

The per-code attempt limit (5) prevents brute-force attacks on the 6-digit code space ($10^6 = 1,000,000$ combinations). With 5 attempts, the probability of a successful brute-force attack is $5/10^6 = 0.0005\%$.

8 Evaluation

8.1 Deployment Setup

The platform was deployed in production for a cohort whose enrolment roster contained 823 users across 7 attribute categories (Table 7). We are deliberate about that wording. The figure 823 is the size of the roster uploaded by the administrator, which bounds who was eligible to register; it is not a count of users who registered, authenticated, joined a group, or completed one. Those counts were not extracted from the deployment before its data was reset, and we therefore do not report them. An earlier version of this paper described the system as “serving 823 users”, which overstates what the number establishes. Groups of exactly 6 members were required, with the following configuration:

- $N = 6$ (required group size)
- $K = 4$ (max from any single category)
- $D = 2$ (minimum unique categories)
- $\mathcal{S} = \{\{\text{ECE}, \text{EEE}\}\}$, that is $q = 1$: at least one member from ECE or EEE
- $\mathcal{B} = \{\text{CSE}, \text{ISE}, \text{ECE}, \text{AI\&ML}, \text{AI\&DS}, \text{EEE}, \text{IoT}\}$

The mandatory representation requirement ($\mathcal{S} = \{\{\text{ECE}, \text{EEE}\}\}$) was chosen because CSE

Table 7: User Distribution by Attribute Category

Code	Category	Users
CSE	Computer Science & Engineering	197
ISE	Information Science & Engineering	194
ECE	Electronics & Communication Eng.	163
AI&ML	AI and Machine Learning	100
AI&DS	AI and Data Science	87
EEE	Electrical & Electronics Eng.	45
IoT	Internet of Things	37
Total		823

and ISE users (47.5% of the cohort) tend to form homogeneous groups, missing the hardware/-electronics perspective. The constraint ensures every completed group includes at least one member with electronics background.

8.2 Performance Metrics

Table 8 reports the measured performance characteristics.

Table 8: Measured Performance Metrics

Metric	Measured Value
SSR page load (Edge CDN)	< 1.5s
OTP email delivery (end-to-end)	2 to 5s
Constraint validation (client-side)	< 50ms
Database document read	< 100ms
Real-time notification delivery	< 200ms
CSV batch import (450 docs)	~3s per batch
JWT signing (HS256)	< 5ms

Client-side constraint validation completes in under 50ms, consistent with the $O(|T| + \sum_j |S_j|)$ complexity analysis (Section 4). Real-time notification delivery under 200ms provides near-instantaneous feedback for group formation events.

8.3 Data Integrity Validation

Table 9 lists the invariant checks we ran against the production data set, together with an explicit statement of what each result does and does not support.

Table 9: Data Integrity Results

Metric	Result
Groups violating $\mathcal{C}_1$ to $\mathcal{C}_4$ on inspection	0
Users appearing in more than one group	0
Registration records referencing a missing group	0
Duplicate pending invitations for one pair	0
Groups observed with more than N members	0

How much these zeros are worth. Less than we previously implied, and the distinction matters enough to spell out.

An earlier version of this paper reported “100% constraint satisfaction for completed groups” as the headline result. We retract it. A group is marked complete only when Algorithm 2 returns no errors, so completed groups satisfy the constraints by construction. Measuring the proportion that do is measuring whether the gate is wired up, and reporting 100% conveys no information about the design. It should never have been presented as an outcome.

The remaining rows are genuine observations rather than tautologies, since nothing in the write path structurally prevents the states they look for. But they are weak evidence, for three reasons. They are a single post-hoc inspection of a final state, not continuous monitoring, so a transient violation that was later overwritten leaves no trace. The deployment is small, and the race described in Section 4 has a window of milliseconds, so the expected number of occurrences at this scale is plausibly below one even if the defect is real. And absence of evidence from an uninstrumented system is close to no evidence at all.

What we should have measured, and did not. The informative quantities here are the ones that vary: how often `canAddMember()` rejected an addition, broken down by which constraint fired; how often a soft warning was resolved before completion rather than abandoned; the distribution of time from group creation to completion; and the behaviour of a deliberate concurrency stress test driving simultaneous joins at a group with one free slot, which is the only one of these that would actually probe the race. None of the four was instrumented. We record this as the principal methodological failure of the work: the system was built to enforce invariants and shipped without the telemetry needed to demonstrate that it had.

8.4 Cost Analysis

Table 10 presents the deployment cost across three user-count tiers.

Table 10: Deployment Cost Analysis (USD/month)

Service	Free Tier	<1K users	<5K users
Hosting (Vercel)	100 GB BW	\$0	\$20
Database (Firestore)	1 GiB, 50K reads/day	\$0	\$5 to 10
Email (Brevo)	300/day	\$0	\$25
Auth (Firebase)	10K/month	\$0	\$0
Domain (optional)	n/a	\$0 to 12/yr	\$0 to 12/yr
Total		\$0	\$50 to 55

For deployments up to ~1,000 users, the platform operates entirely within free tiers. Even at 5,000 users, the total cost (\$50 to 55/month) is orders of magnitude lower than commercial solutions.

8.5 Comparison with Web Collaboration Frameworks

Table 11 positions our design against five families of alternative approach across 11 criteria.

Methodology, and its limits. Each cell records whether the stated capability is part of the approach’s documented design intent, as described by its own vendor or author documentation consulted in April 2026. No system other than ours was installed, configured, or executed, and no quantity in this table was measured. It is included only to locate our design in a space of alternatives, and it cannot support a claim that one approach outperforms another.

An earlier version of this paper additionally contained a bar chart assigning each approach a score between 0 and 100 on four dimensions. We have removed it. Those numbers had

Table 11: Design intent of five approaches to group formation, compiled from public documentation. This is a qualitative positioning aid, not a measurement: see the methodology note below.

Criterion		Man.	CSP	LMS	Chat	RTDB	Ours
<i>User experience</i>							
Users	choose members	✗	✗	✓	✓	✓	✓
Feedback	at click time	✗	✗	✗	✗	✗	✓
In-app	notifica-tions	✗	✗	✓	✓	✓	✓
<i>Composition rules</i>							
Rules	expressible	✗	✓	✗	✗	✗	✓
Rules	admin-configurable	✗	✓	✗	✗	✗	✗
Checked	before write	✗	n/a	✗	✗	✗	✓
Atomic	under concurrency	✗	n/a	✗	✗	✗	✗
<i>Administration</i>							
Phased	access gates	✗	✗	✓	✗	✗	✓
Bulk	roster im-port	✗	✓	✓	✗	varies	✓
<i>Deployment</i>							
No	institutional SSO	✓	✗	✗	✓	✓	✓
Source	available	✗	varies	✓	✗	✗	✓

Man. = manual allocation; CSP = batch constraint solver; LMS = learning management system group tools; Chat = team chat platforms; RTDB = a realtime document store used directly from the client.

no derivation: they were not measurements, they were not the output of a rubric applied by independent raters, and they were not reproducible by anyone including us. Presenting them on a numeric axis implied a precision that did not exist, and both reviewers of the earlier version were right to ask how they were obtained. The two rows in this table where our own system scores **X** are also worth noting explicitly, since a comparison that flatters the authors on every line is not a comparison: our composition rules are not administrator-configurable, and our member additions are not atomic under concurrency.

The pattern in the table is a trade-off rather than a ranking. Batch constraint solvers express composition rules well and remove user choice entirely, which for our cohort was the property we were asked to avoid. Learning management systems and team chat platforms preserve choice and offer no rule enforcement. Using a realtime document store directly from the client gives live updates but leaves every invariant to client code, which is untrustworthy by construction. Our design keeps choice and adds enforcement at click time, and pays for it in the two rows where it fails: the rules are compiled in rather than configurable, and additions are not atomic.

8.6 Implementation Statistics

Table 12 summarises the implementation, counted with `wc -l` over `src/` at commit `e0bc21d` of the public repository so that the figures can be reproduced. An earlier version of this paper reported 8,245 lines as the total. That figure is the `.tsx` subtotal; the total across all 58 TypeScript source files is 10,937. We report both here to make the correction explicit.

Table 12: Implementation Statistics

Metric	Value
Total TypeScript lines of code	10,937
in 31 <code>.tsx</code> files	8,245
in 27 <code>.ts</code> files	2,692
Source files	58
React components	19
API endpoints	11
Utility/library modules	14
Type definitions	2 files (71+ types)
Framework	Next.js 16 (App Router)
Database	Firebase Firestore
Authentication	Firebase Auth + JWT
Hosting	Vercel (serverless)
Email service	Brevo (transactional API)
Firestore collections	7
Artifact commit	<code>e0bc21d</code>

9 Discussion

9.1 Constraint Architecture Effectiveness

The production deployment validates that real-time constraint enforcement is practically viable on serverless NoSQL infrastructure. The two-phase validation algorithm (Algorithms 1 and 2) achieves the critical balance: users retain full autonomy in member selection while the system refuses to mark a group complete unless the constraints hold. We no longer describe that as a measured outcome, for the reason given in Section 8: completion is gated on the predicate, so the relationship is definitional.

The progressive warning system proved effective in practice. Rather than imposing hard blocks early (which would frustrate users during the exploratory phase of group building), the system guides users toward constraint-satisfying compositions through timely warnings. This design respects human decision-making processes, users receive constraint status information at the point when it becomes actionable.

9.2 Revisiting the Decision to Avoid Transactions

The framing of this work in its earlier form, that comprehensive integrity is achievable on document-store infrastructure “without multi-document transactions”, invites an obvious objection: Firestore has supported multi-document transactions for years [42], so avoiding them is not an achievement to be reported but a constraint to be justified, and we did not justify it. We accept the objection.

It is worth separating the part of the framework that survives this concession from the part that does not. Five of the six layers concern invariants that are enforced by document structure or by a uniqueness property of a document key: embedding the member list inside the group document makes a member addition a single-document update, and deriving an invitation’s document identifier from the pair it concerns makes a duplicate invitation unrepresentable rather than merely detectable. Those layers are not substitutes for transactions and do not compete with them. Choosing a data model in which an invariant cannot be expressed in an invalid state is good design under any consistency regime, and we would keep all five.

The sixth layer is the one that was doing work transactions should have done. Guarding a capacity constraint by re-reading before writing is an attempt to approximate atomicity with ordering, and it does not succeed, for the reason set out in Section 4. The defensible reading of our experience is therefore narrower than our original claim and points the other way: application-level defence in depth is an excellent way to make invalid states unrepresentable, and a poor way to make a read-modify-write sequence atomic. For the latter there is a correct mechanism, it was one function call away, and we did not use it.

What we did subsequently. The same authors later built a second platform on the same stack, for event registration with capacity limits, which is structurally the same problem: a bounded counter under concurrent contention. In that system the capacity check and the counter increment are performed inside `runTransaction` with an atomic field increment, and the read-before-write pattern described here does not appear. We did not adopt transactions because of an analysis; we adopted them because the pattern in this paper felt uncomfortable to maintain under a second set of requirements. Readers may reasonably treat that as the most useful empirical finding in this report.

9.3 The Email Quota Arrangement in Practice

The rotation delivered the registration burst without a user-visible failure, and we originally reported this as evidence that it worked. Two qualifications now seem more important than the result.

The first is the one developed in Section 5.5: the arrangement should not have been built, because it summed the allowances of several free accounts, and we describe the alternatives there. The second is that we cannot substantiate the load distribution we claimed. We asserted that each of three credentials carried approximately a third of the traffic, but that figure was inferred from the design of the counter rather than observed, and the counter resides in per-instance memory on a runtime that recycles instances, so on each cold start selection restarts at the first credential. Per-credential delivery counts were available in the provider’s dashboard at the time and we did not record them. The honest statement is that the burst was delivered and that we do not know how it was distributed.

9.4 Serverless Trade-offs

The serverless architecture provides three significant advantages: zero-cost deployment on free tiers, automatic scaling, and zero operational maintenance. However, it introduces two trade-offs:

1. **In-memory state volatility:** The rate limiter and round-robin counter store state in server memory, which resets on cold starts. For the rate limiter, this means a brief window after cold start where rate limits are not enforced; for the round-robin counter, this means temporarily non-uniform key selection. Neither affects correctness, only best-effort quality.
2. **Cold-start latency:** The first request after an idle period incurs cold-start overhead (typically 200 to 500ms on Vercel). This is acceptable for our use case where sub-second response times are sufficient, but would be problematic for latency-sensitive real-time applications.

9.5 Limitations

1. **Cold-start problem:** Early registrants have fewer groups to browse; the platform is most effective when a critical mass of users has registered before group formation opens. The dual-gate system mitigates this.
2. **In-memory rate limiting:** The current rate limiter does not persist across serverless function instances. For high-traffic deployments, a distributed store (e.g., Redis) would provide cross-instance persistence.
3. **Constraint flexibility:** While the four-parameter model covers common diversity requirements, adding entirely new constraint *types* (e.g., GPA distribution, schedule compatibility) requires code modifications.
4. **Eventual consistency edge case:** While the read-before-write pattern prevents most race conditions, extremely rare simultaneous writes (within the same millisecond) could theoretically produce inconsistent state. This was not observed in production.
5. **Non-atomic member addition:** The read-before-write guard does not make the capacity check atomic, so a concurrent pair of additions can in principle exceed N . This is a defect, not a trade-off, and its remedy is a multi-document transaction (Section 9).
6. **Absence of telemetry:** The deployment recorded no constraint-rejection counts, warning-resolution rates, or per-credential delivery counts, and ran no concurrency stress test. Several claims in the earlier version of this paper rested on quantities that were never instrumented, and we have withdrawn them rather than restate them.
7. **Parameters are not configurable:** N , K , D and the requirement group are compile-time constants in the released artifact, so the reconfigurability the model suggests is not available to an administrator.
8. **Model and implementation drift:** The published constraint model stated a conjunctive mandatory-representation rule while the code enforced a disjunctive one. The formalism did not catch this because nothing connected the two; a property-based test generated from the model would have.
9. **Single deployment, and limits on what it can support:** Validation was performed at one institution, with one cohort, one set of parameter values, and one requirement group of size two. We have removed the claim that the architecture is domain-agnostic. Nothing here establishes behaviour at a different scale, under a different constraint shape, or with $q > 1$, and the feasibility theorem’s disjointness hypothesis has never been exercised non-trivially.

10 Future Work

1. **Distributed rate limiting:** Replace the in-memory rate limiter with a Redis-backed implementation that persists across serverless function invocations and supports horizontal scaling. This would eliminate the cold-start rate-limit gap and enable consistent enforcement across multiple function instances.
2. **Instrumentation and a concurrency stress test:** Add the telemetry this deployment lacked (rejections by constraint, warning resolution, time to completion) and drive a deliberate stress test of simultaneous joins against a group with one free slot, both before and after converting the capacity guard to a transaction, so that the cost of the correct mechanism is measured rather than assumed.
3. **Constraint DSL:** Develop a domain-specific language allowing administrators to define arbitrary constraint rules (e.g., “at most 2 from same subcategory AND GPA variance < 0.5 ”) via a web interface without code changes.
4. **GraphQL API:** Extend the REST API with a GraphQL layer enabling fine-grained data fetching, reducing over-fetching on the browse interface, and enabling client-driven query composition for diverse frontend requirements.
5. **Formal verification:** Apply model checking [49] to verify the six-layer data integrity framework against a formal specification of the invariant set, providing machine-checked correctness guarantees beyond testing-based validation.
6. **Multi-region deployment:** Extend the architecture for multi-region deployment with cross-region data replication, investigating the impact of geographic distribution on constraint enforcement latency and consistency guarantees.
7. **Adaptive rate limiting:** Implement adaptive rate-limit policies that dynamically adjust thresholds based on observed traffic patterns, reducing false positives during legitimate bursts while maintaining protection against abuse.
8. **Real-time constraint visualization:** Develop interactive constraint satisfaction visualizations showing users which group compositions are feasible given current membership, enabling more informed member selection.

11 Conclusion

This paper reported our experience building, deploying, and subsequently re-examining a platform for constrained self-service group formation on serverless document-store infrastructure. The artifact is real, was used by a live cohort, and is open for inspection at a fixed commit; that, rather than novelty of mechanism, is what we offer.

Three things are worth carrying away. The first is the composition model of Section 4, whose mandatory-representation constraint is disjunctive because that is what real policies look like, and whose feasibility conditions reduce to three independent inequalities. Stating them forced us to find an error in our own earlier formulation, where a condition we had asserted as necessary was not, on the strength of a proof that misread a lower bound as an upper one. We would not have found it without a reviewer’s observation that the condition was redundant.

The second is a boundary. Enforcing composition rules at the instant of user action is cheap, because the group is small and the check is a linear scan, and it materially changes what the interface can offer: a user learns that an addition is inadmissible before making it rather than after. Making that enforcement correct under concurrency is a separate problem, which the interface work does not solve and which we did not solve either.

The third is the negative result, and it is the part we would most want a practitioner to read. We avoided multi-document transactions that the database provided, guarded a capacity invariant with a read-before-write pattern that narrows a race without closing it, claimed a correctness guarantee that pattern does not deliver, and reported an absence of violations from a

system with no means of detecting them. We compounded this by presenting as a headline result a figure, 100% constraint satisfaction among completed groups, which is true by construction. Separately, we multiplied an email quota across free accounts and described it as a contribution when it was a circumvention of the provider’s pricing; we withdraw it and give the sanctioned alternatives in Section 5.5. A later platform by the same authors, facing the same bounded-counter problem, uses transactions.

The generalisable lesson is not about document stores. It is that a formal model and a deployed implementation drift apart unless something forces them together, and that a system built to maintain invariants should ship with the instrumentation needed to show that it did. We had the first without the second, and consequently our strongest claims were the ones we could least support.

Acknowledgments

The authors thank the Principal and Management of Don Bosco Institute of Technology, Bangalore, for their support and encouragement.

Declarations

- **Funding:** This research received no external funding.
- **Conflict of interest:** The authors declare that they have no conflict of interest.
- **Ethics approval:** Not applicable. The study used anonymized, aggregate deployment data.
- **Consent to participate:** Not applicable.
- **Consent for publication:** Not applicable.
- **Data availability:** The source code and documentation are publicly available at <https://github.com/harshaxyZ/idea.lab>. The production deployment data (anonymized user statistics) are available from the corresponding author upon reasonable request.
- **Code availability:** Available at <https://github.com/harshaxyZ/idea.lab> under MIT license.
- **Author contribution:** All authors contributed to the study conception and design. Material preparation, data collection and analysis were performed by Mithun Gowda B and Harsha N. The first draft of the manuscript was written by Mithun Gowda B and all authors commented on previous versions of the manuscript. All authors read and approved the final manuscript.

References

- [1] Cattell R (2010) Scalable SQL and NoSQL data stores. ACM SIGMOD Record 39(4):12–27. <https://doi.org/10.1145/1978915.1978919>
- [2] Vogels W (2009) Eventually consistent. Communications of the ACM 52(1):40–44. <https://doi.org/10.1145/1435417.1435432>
- [3] Han J, Haihong E, Le G, et al (2011) Survey on NoSQL database. Proc 6th Int Conf Pervasive Computing and Applications pp 363–366. <https://doi.org/10.1109/ICPCA.2011.6106531>
- [4] Pimentel V, Nickerson BG (2012) Communicating and displaying real-time data with Web-Socket. IEEE Internet Computing 16(4):45–53. <https://doi.org/10.1109/MIC.2012.64>

- [5] Lubbers P, Greco F (2010) HTML5 WebSocket: A quantum leap in scalability for the web. SOA World Magazine URL <https://websocket.org/quantum.html>
- [6] Wang V, Salim F, Moskovits P (2013) The definitive guide to HTML5 WebSocket. Apress <https://doi.org/10.1007/978-1-4302-4741-8>
- [7] Rossi F, van Beek P, Walsh T (2006) Handbook of Constraint Programming. Foundations of Artificial Intelligence, Elsevier
- [8] Kumar V (1992) Algorithms for constraint-satisfaction problems: A survey. AI Magazine 13(1):32–44. <https://doi.org/10.1609/aimag.v13i1.976>
- [9] Brewer EA (2000) Towards robust distributed systems. In: Proc. 19th Annual ACM Symp. Principles of Distributed Computing (PODC), <https://doi.org/10.1145/343477.343502>, keynote address. Introduced the CAP conjecture.
- [10] Gilbert S, Lynch N (2002) Brewer’s conjecture and the feasibility of consistent, available, partition-tolerant web services. ACM SIGACT News 33(2):51–59. <https://doi.org/10.1145/564585.564601>
- [11] DeCandia G, Hastorun D, Jampani M, et al (2007) Dynamo: Amazon’s highly available key-value store. ACM SIGOPS Operating Systems Review 41(6):205–220. <https://doi.org/10.1145/1323293.1294281>
- [12] Jonas E, Pu Q, Venkataraman S, et al (2017) Occupy the cloud: Distributed computing for the 99%. In: Proc. 2017 Symp. Cloud Computing (SoCC). ACM, pp 445–451, <https://doi.org/10.1145/3127479.3128601>
- [13] Baldini I, Castro P, Chang K, et al (2017) Serverless computing: Current trends and open problems. Research Advances in Cloud Computing pp 1–20. https://doi.org/10.1007/978-981-10-5026-8_1
- [14] Schleier-Smith J, Sreekanti V, Khandelwal A, et al (2021) What serverless computing is and should become: The next phase of cloud computing. Communications of the ACM 64(5):76–84. <https://doi.org/10.1145/3406011>
- [15] Bafoutsou G, Mentzas G (2002) Review and functional classification of collaborative systems. Int Journal of Information Management 22(4):281–305. [https://doi.org/10.1016/S0268-4012\(02\)00013-0](https://doi.org/10.1016/S0268-4012(02)00013-0)
- [16] Olson GM, Olson JS (2000) Distance matters. Human-Computer Interaction 15(2–3):139–178. https://doi.org/10.1207/S15327051HCI1523_4
- [17] Grigorik I (2013) High Performance Browser Networking. O’Reilly Media, chapters on SSE, WebSocket, and HTTP/2
- [18] Google (2026) Firebase documentation. URL <https://firebase.google.com/docs>
- [19] Lappas T, Liu K, Terzi E (2009) Finding a team of experts in social networks. In: Proc. 15th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining. ACM, Paris, France, pp 467–476, <https://doi.org/10.1145/1557019.1557074>
- [20] Anagnostopoulos A, Becchetti L, Castillo C, et al (2012) Online team formation in social networks. In: Proc. 21st Int. Conf. World Wide Web (WWW). ACM, Lyon, France, pp 839–848, <https://doi.org/10.1145/2187836.2187950>

- [21] Sun Y, Deng X, Papageorgiou DG (2026) Two-phase simulated annealing for equitable team formation: Eliminating complaints in large engineering cohorts. URL <https://arxiv.org/abs/2606.07270>, arXiv preprint arXiv:2606.07270
- [22] Wi H, Oh S, Mun J, et al (2009) A team formation model based on knowledge and collaboration. *Expert Systems with Applications* 36(5):9121–9134. <https://doi.org/10.1016/j.eswa.2008.12.031>
- [23] Tsang EPK (1999) A glimpse of constraint satisfaction. *Artificial Intelligence Review* 13(3):215–227. <https://doi.org/10.1023/A:1006558104682>
- [24] Stonebraker M (2010) SQL databases v. NoSQL databases. *Communications of the ACM* 53(4):10–11. <https://doi.org/10.1145/1721654.1721659>
- [25] Chang F, Dean J, Ghemawat S, et al (2006) Bigtable: A distributed storage system for structured data. In: *Proc. 7th USENIX Symp. Operating Systems Design and Implementation (OSDI)*, pp 205–218
- [26] Jonas E, Schleier-Smith J, Sreekanti V, et al (2019) Cloud programming simplified: A Berkeley view on serverless computing. Technical Report No UCB/EECS-2019-3 URL <https://www2.eecs.berkeley.edu/Pubs/TechRpts/2019/EECS-2019-3.html>
- [27] McGrath G, Brenner PR (2017) Serverless computing: Design, implementation, and performance. In: *Proc. IEEE 37th Int. Conf. Distributed Computing Systems Workshops (ICDCSW)*, pp 405–410, <https://doi.org/10.1109/ICDCSW.2017.36>
- [28] Tanenbaum AS, Wetherall DJ (2011) *Computer networks*. Pearson Covers token bucket and leaky bucket algorithms for traffic shaping and rate limiting
- [29] Polli R, Martinotti A (2024) RateLimit header fields for HTTP. IETF Internet-Draft, draft-ietf-httpapi-ratelimit-headers, URL <https://datatracker.ietf.org/doc/draft-ietf-httpapi-ratelimit-headers/>
- [30] Raghavan B, Vishwanath K, Ramabhadran S, et al (2007) Cloud control with distributed rate limiting. In: *Proc. ACM SIGCOMM*, pp 337–348, <https://doi.org/10.1145/1282380.1282419>
- [31] van der Aalst WMP, ter Hofstede AHM, Weske M (2003) Business process management: A survey. *Business Process Management, LNCS* 2678:1–12. https://doi.org/10.1007/3-540-44895-0_1
- [32] Harel D (1987) Statecharts: A visual formalism for complex systems. *Science of Computer Programming* 8(3):231–274. [https://doi.org/10.1016/0167-6423\(87\)90035-9](https://doi.org/10.1016/0167-6423(87)90035-9)
- [33] Hopcroft JE, Karp RM (1971) A linear algorithm for testing equivalence of finite automata. Cornell University Technical Report Foundation for FSM-based workflow modeling
- [34] OWASP Foundation (2021) OWASP top ten web application security risks. URL <https://owasp.org/www-project-top-ten/>
- [35] Barth A, Jackson C, Mitchell JC (2008) Robust defenses for cross-site request forgery. In: *Proc. 15th ACM Conf. Computer and Communications Security (CCS)*, pp 75–88, <https://doi.org/10.1145/1455770.1455782>
- [36] Jones M, Bradley J, Sakimura N (2015) JSON Web Token (JWT). IETF RFC 7519, URL <https://datatracker.ietf.org/doc/html/rfc7519>

- [37] Hardt D (2012) The OAuth 2.0 authorization framework. IETF RFC 6749, URL <https://datatracker.ietf.org/doc/html/rfc6749>
- [38] Stuttard D, Pinto M (2011) The Web Application Hacker’s Handbook, 2nd edn. Wiley
- [39] Zalewski M (2012) The tangled web: A guide to securing modern web applications
- [40] M’Raihi D, Machani S, Pei M, et al (2011) TOTP: Time-based one-time password algorithm. IETF RFC 6238, URL <https://datatracker.ietf.org/doc/html/rfc6238>
- [41] Karp RM (1972) Reducibility among combinatorial problems. In: Miller RE, Thatcher JW (eds) Complexity of Computer Computations. Plenum Press, p 85–103, https://doi.org/10.1007/978-1-4684-2001-2_9
- [42] Google (2026) Transactions and batched writes. URL <https://firebase.google.com/docs/firestore/manage-data/transactions>, cloud Firestore documentation. Multi-document transactions use optimistic concurrency control and retry on contention
- [43] Google Cloud (2019) Cloud Firestore data model and consistency guarantees. URL <https://firebase.google.com/docs/firestore/manage-data/transactions>, firestore provides strong consistency for document reads and queries within a single document
- [44] Lamport L (1979) How to make a multiprocessor computer that correctly executes multiprocess programs. IEEE Transactions on Computers C-28(9):690–691. <https://doi.org/10.1109/TC.1979.1675439>
- [45] Vercel (2026) Next.js documentation. URL <https://nextjs.org/docs>
- [46] Fielding RT (2000) Architectural styles and the design of network-based software architectures. PhD thesis, University of California, Irvine, introduced REST architectural style
- [47] Richardson L, Ruby S (2007) RESTful web services. O’Reilly Media
- [48] Brevo (2026) What are the limits of the free plan? URL <https://help.brevo.com/hc/en-us/articles/208580669>, help centre article. The free plan permits 300 emails per day per account; higher allowances require a paid plan, and administrator sub-accounts draw credits from a single purchased pool
- [49] Baier C, Katoen JP (2008) Principles of Model Checking. MIT Press